

The only positive prime value occurs at index 2

A disproof of conjecture 00000007662

ColdSmile

October 2026

Abstract

For the formal series defined by the equation in the conjecture, we prove that $C_n(2)$ is a positive prime precisely when $n = 2$, with $C_2(2) = 2$. The prime-index set is therefore finite, contradicting the explicit infinitude assertion. A Lean 4 project verifies the full argument, including existence and uniqueness of the original formal series and the polynomial evaluation at $q = 2$.

1 The stated equation and its actual coefficients

Both language versions of conjecture 00000007662 define the coefficients by

$$F(q, t) = \frac{1 - qtF(q, qt)}{1 - tF(q, t)} \quad (1)$$

and assert that infinitely many indices n have $C_n(2)$ prime, together with upper and lower bounds for the number of such indices up to x . We use the displayed equation literally: its minus sign and the argument t in the denominator are retained. Primes mean ordinary positive primes. The index set is $\mathbb{N} = \{0, 1, 2, \dots\}$; starting at 1 makes no difference.

Work first in $\mathbb{Z}[Q][[t]]$. More generally, in any commutative ring R with $q \in R$, the constant term of $1 - tF$ is 1, so it is invertible. Thus (1) is equivalent to

$$F = 1 + t(F^2 - qF(qt)). \quad (2)$$

Here the coefficient of t^n in $F(qt)$ is q^n times that in $F(t)$. Consequently the coefficient sequence is exactly the triangular recursion

$$a_0(q) = 1, \quad a_{n+1}(q) = \sum_{i=0}^n a_i(q)a_{n-i}(q) - q^{n+1}a_n(q). \quad (3)$$

Every index on the right is smaller than $n + 1$. This constructs a unique sequence and a unique formal series. Conversely, the series with these coefficients satisfies (2) coefficientwise; invertibility of $1 - tF$ then gives (1). In $\mathbb{Z}[Q]$, each coefficient is a polynomial. Applying evaluation $Q \mapsto 2$ to the finite sums and products in (3) gives the integer sequence $a_n = C_n(2)$ with

$$a_0 = 1, \quad a_{n+1} = \sum_{i=0}^n a_i a_{n-i} - 2^{n+1} a_n.$$

No analytic convergence or unproved hypothesis about existence is used.

2 A positive transform and uniform growth

Define integers b_n by

$$b_0 = 1, \quad b_{n+1} = 2^{n+1}b_n - \sum_{i=0}^n b_i b_{n-i}. \quad (4)$$

We prove positivity and growth; neither is imposed as an assumption.

Lemma 1. *Fix n . Suppose $u_0 = 1$, $u_k \geq 0$ for $k \leq n$, and for every $k < n$,*

$$2^k u_k \leq u_{k+1} \leq 2^{k+1} u_k.$$

Then $u_i u_j \leq u_{i+j}$ whenever $i + j \leq n$.

Proof. Induct on i . The case $i = 0$ is equality. At the induction step, $j = 0$ is also equality. For $j \geq 1$ and $i + 1 + j \leq n$, all bounds below have index less than n , and nonnegativity permits multiplication:

$$u_{i+1} u_j \leq 2^{i+1} u_i u_j \leq 2^{i+1} u_{i+j} \leq 2^{i+j} u_{i+j} \leq u_{i+j+1}.$$

The middle comparison uses $i + 1 \leq i + j$. □

Lemma 2. *For every $n \geq 0$,*

$$b_n \geq 1, \quad 2^n b_n \leq b_{n+1} \leq 2^{n+1} b_n. \quad (5)$$

Proof. Use strong induction on n , proving all three assertions together. If $n = 0$, $b_n = 1$. If $n = k + 1$, the earlier lower bound and $b_k \geq 1$ give $b_n \geq 2^k b_k \geq 1$. Thus $b_i \geq 0$ for all $i \leq n$. All earlier growth bounds are now available, so Lemma 1 gives $b_i b_{n-i} \leq b_n$. Hence

$$0 \leq \sum_{i=0}^n b_i b_{n-i} \leq (n+1) b_n \leq 2^n b_n.$$

The elementary bound $n + 1 \leq 2^n$ follows by induction starting at $n = 0$. Subtract the displayed convolution bounds from $2^{n+1} b_n$ in (4) to obtain the two required growth bounds at n . This proves the induction without using the bounds at n prematurely. □

In particular, for $n \geq 3$ the lower bound at $n - 1$ gives

$$b_n \geq 2^{n-1} b_{n-1} \geq 4. \quad (6)$$

3 Signs, parity, and the exact prime-index set

Lemma 3. *For every n , $a_n = (-1)^n b_n$. Thus $a_n < 0$ for odd n , while $a_n = b_n \geq 1$ for even n .*

Proof. The case $n = 0$ is immediate. Assuming the formula at all indices at most n , every product in the convolution acquires the same factor $(-1)^n$. Equation (3) at $q = 2$ therefore yields

$$a_{n+1} = (-1)^n \left(\sum_{i=0}^n b_i b_{n-i} - 2^{n+1} b_n \right) = (-1)^{n+1} b_{n+1}.$$

The sign assertions follow from Lemma 2. □

Lemma 4. *Every coefficient with a positive even index is even.*

Proof. Write the index as $2k+2$. Reflection $i \mapsto 2k+1-i$ pairs the terms of the preceding odd-index convolution with no fixed point, giving

$$\sum_{i=0}^{2k+1} a_i a_{2k+1-i} = 2 \sum_{i=0}^k a_i a_{2k+1-i}.$$

In (3), this even sum is reduced by $2^{2k+2} a_{2k+1}$, also even. □

Theorem 1. *For the coefficients of (1), evaluated at $q = 2$,*

$$C_n(2) \text{ is a positive prime} \iff n = 2.$$

Proof. The first three values from (3) are $a_0 = 1$, $a_1 = -1$, and $a_2 = 2$. Odd indices have negative values by Lemma 3. For even $n \geq 4$, Lemma 4 gives an even value, and (6) gives $a_n = b_n \geq 4$. Such a value cannot be a positive prime, because the only even positive prime is 2. This covers every index. □

It follows that the prime-index set is exactly $\{2\}$. This contradicts the explicit infinitude conjunct, and therefore disproves the conjecture as a whole. For real $x \geq 0$ its counting function is exactly

$$A(x) = \#\{n \leq x : C_n(2) \text{ is a positive prime}\} = \begin{cases} 0, & x < 2, \\ 1, & x \geq 2. \end{cases}$$

Thus the asserted lower bound cannot hold under either the eventual lower-bound or the infinitely-often convention for $\Omega(\log \log x)$. The disproof relies on the false infinitude assertion and does not require formalizing either asymptotic convention or refuting the upper bound.

4 Formalization and reproducibility

The accompanying Lean project pins Lean 4.19.0 and Mathlib v4.19.0. Its objects are actual formal power series, rather than a recurrence given the name of the conjectured sequence. In namespace `Conjecture7662`, the principal correspondence is:

Lean declaration	Mathematical role
<code>FunctionalEquation</code>	Exact quotient equation, with unit inverse
<code>functionalEquation_coefficients</code>	Equivalence with the coefficient recursion
<code>exists_unique_series</code>	Existence and uniqueness over a commutative ring
<code>polynomial_evaluation</code>	$\mathbb{Z}[Q]$ coefficients specialize to a_n
<code>b_growth, coefficient_signed</code>	Growth bounds and alternating sign
<code>even_index_coefficient</code>	Evenness at every positive even index
<code>positivePrime_iff</code>	Exact prime classification of a_n
<code>formal_series_disproof</code>	Unique existence and failure of infinitude for every actual solution
<code>polynomial_prime_iff</code>	The same classification for $C_n(2)$
<code>primeCounting_eq</code>	Exact count for every real cutoff

The predicate `PositivePrime(z)` is $\exists p \in \mathbb{N}, \text{Prime}(p) \wedge z = (p : \mathbb{Z})$. It does not take an absolute value or truncate negative coefficients. The final disproof has no additional growth, sign, or primality hypotheses.

The standalone auxiliary Python checker compares two exact integer methods: the triangular recursion and iteration of the original quotient, using truncated formal-series inversion and substitution $t \mapsto qt$. It checks all coefficients through degree 32 for $q \in \{-2, -1, 0, 1, 2, 3, 4\}$ (231 comparisons) and verifies each truncated quotient identity. The beginning at $q = 2$ is

$$1, -1, 2, -11, 150, -4474.$$

These finite checks provide implementation cross-checks; the all-index mathematical result is the Lean theorem, not an extrapolation of the tests.

From the submitted `lean/` directory, reproduce the proof with

```
lake exe cache get
lake build
lake env lean -DwarningAsError=true Conjecture7662.lean
lake env lean -DwarningAsError=true Check.lean
```

The package README gives the auxiliary command and describes all evidence. Verification records include a fresh independent project build, strict source replay, declared theorem axioms, a complete inventory of compiled module declarations, pinned dependency identities, and matching report source/PDF checks. These are local checks; acceptance belongs to the competition maintainers.

Source. The official bilingual statement is included byte-for-byte as `conjecture.md` and is available in the repository at `conjectures/00000007662.md`. The argument above is derived directly from that displayed equation.